

FIAP - FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO  
PAULISTA  
DEFESA CIBERNÉTICA



CHALLENGE FASE 4  
HAKAI

SÃO PAULO  
2025

# SUMÁRIO

## INTRODUÇÃO

- 1.1. Contexto do Desafio: A Evolução da Prova de Conceito (PoC)

## DESENVOLVIMENTO: ARQUITETURA E ANÁLISE TÉCNICA COMPARATIVA

- 2.1. Reformulação Arquitetural da Aplicação (UI/UX)
- 2.2. Implementação do Portal de Demonstração (Ref. DOCX 5.2)
  - 2.2.1. Página de Produto e Modal de Contexto
  - 2.2.2. Implementação Técnica de Download de Artefato
- 2.3. Simulação de Provisão de Credenciais 2FA (Interface "Nexshop Mail")
- 2.4. Implementação do Fluxo de Registro de Novo Usuário
- 2.5. Implementação de Gerenciamento de Estado e Lógica de Validação Dupla
- 2.6. Análise de Correção Crítica: Validação EQUIPE

## CONCLUSÃO TÉCNICA

**Link**

# . INTRODUÇÃO

## 1.1. Contexto do Desafio: A Evolução da Prova de Conceito (PoC)

O artefato de linha de base (en5.html) representava uma Prova de Conceito (PoC) linear, demonstrando um fluxo de autenticação em múltiplas etapas para um único usuário predefinido ("Samuel"). Embora funcional para validar a sequência de telas, esta versão carecia de interatividade, fluxos de usuário alternativos e funcionalidades estratégicas delineadas no documento-guia do projeto (

challenge.2.docx).

O objetivo desta evolução técnica foi transformar o artefato base de um script de demonstração passivo em um portal interativo e multifacetado (

challenge 4.html), que não apenas endereça os requisitos de demonstração de forma mais robusta, mas também implementa ativamente objetivos de projeto futuros, como a criação de uma página para download de artefatos.

Este relatório detalha as mudanças técnicas e arquiteturais específicas na transição do en5.html para o challenge 4.html.

## 2. DESENVOLVIMENTO: ARQUITETURA E ANÁLISE TÉCNICA COMPARATIVA

### 2.1. Reformulação Arquitetural da Aplicação (UI/UX)

A mudança mais fundamental foi a refatoração da arquitetura DOM.

- **Versão Base (en5.html):** A aplicação consistia em um único contêiner visível (#main-container) que continha todas as "seções" (login, 2fa, etc.), iniciando diretamente na tela de login.
- **Versão Final (challenge 4.html):** A estrutura foi promovida para uma aplicação de múltiplas "visões" de nível superior, mutuamente exclusivas:
  1. #download-section: O novo ponto de entrada da aplicação, visível por padrão.
  2. #app-wrapper: O #main-container original foi encapsulado neste novo *wrapper*, que agora fica oculto (hidden) por padrão e contém todo o fluxo de autenticação (login, registro, 2FA, etc.).
  3. #gmail-section: Um novo contêiner de tela cheia, também oculto, que simula um serviço externo.

Esta nova arquitetura permite uma separação clara entre a "página do produto" e o "aplicativo" em si, exigindo a refatoração da navegação e do gerenciamento de estado (detalhado na seção 2.5).

## 2.2. Implementação do Portal de Demonstração (Ref. DOCX 5.2)

Alinhado diretamente com a seção 5.2 do documento de referência ("Criação de Página para Download de Artefatos de Demonstração"), a versão final implementa este requisito por completo.

- **2.2.1. Página de Produto e Modal de Contexto** O #download-section atua como a página de download e produto. Seu layout foi estilizado com TailwindCSS para simular uma página de produto moderna, com descrições e CTAs claros. Para fornecer contexto adicional, um modal (#modal-section) foi criado e ativado pelo novo botão #saber-mais-button, fornecendo um resumo sobre o projeto NexShop Anti-Fraude.
- **2.2.2. Implementação Técnica de Download de Artefato** O link "Baixar Módulo" (

#download-module-link) foi movido da proposta conceitual para a implementação funcional. Na carga da página (evento

DOMContentLoaded), um script captura todo o conteúdo HTML da página atual via `document.documentElement.innerHTML`. Este conteúdo é então codificado para um formato URL-safe usando `encodeURIComponent()` e prepõe o MIME type e charset (`data:text/html;charset=utf-8,`). O Data URL resultante é atribuído dinamicamente ao atributo `href` do link. Combinado com o atributo `download="index.html"`, isso permite que o usuário baixe o próprio código-fonte da aplicação que está utilizando.

Exemplo: Nessa Parte do Baixar módulo pode-se baixar esse próprio index para teste

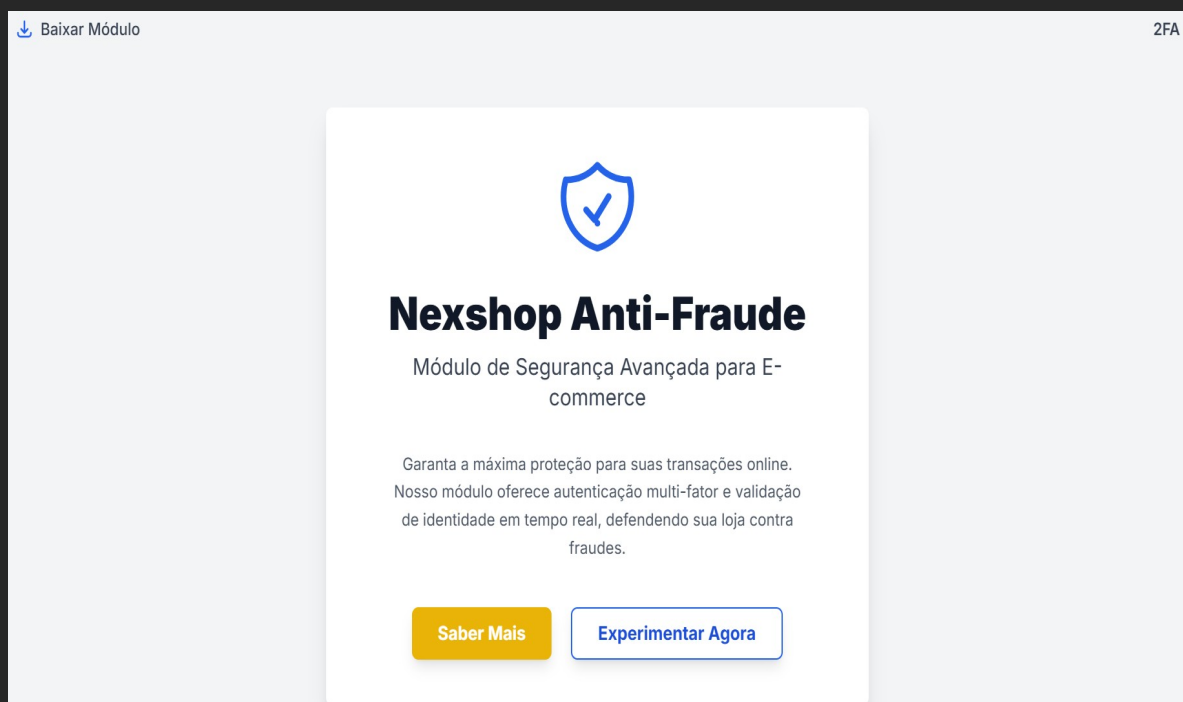


Figura 1 Novos recursos de web

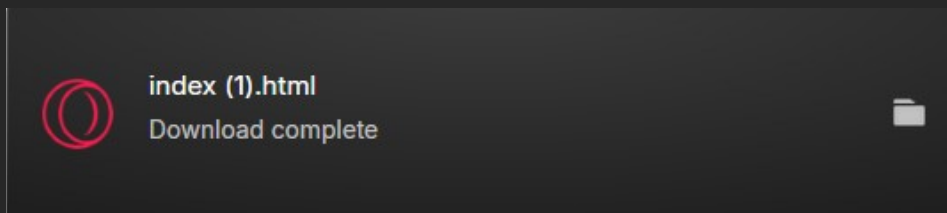


Figura 2 Index Baixado

### 2.3. Simulação de Provisão de Credenciais 2FA (Interface "Nexshop Mail")

Para simular o processo de envio de código 2FA (mencionado conceitualmente no DOCX ) sem um backend real, a interface

#gmail-section foi criada.

- Esta seção é uma simulação de cliente de e-mail em tela cheia, acessada pelo novo botão "2FA" no cabeçalho do #download-section.
- Graficamente, a logo do Google foi substituída por um logo textual (<span class="...">Nexshop Mail</span>), conforme solicitado.
- Funcionalmente, ela serve como um repositório para os códigos de validação. A interface gerencia seu próprio estado interno (lista de e-mail vs. visualização de e-mail) através de event listeners nos e-mails (#email-1, #email-2) e nos botões de retorno (.gmail-back-to-inbox). Isso permite ao usuário "consultar" ativamente os códigos samuel e EQUIPE em um ambiente simulado.

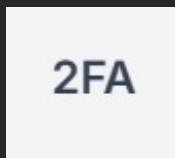


Figura 3 Direciona para nexshop mail

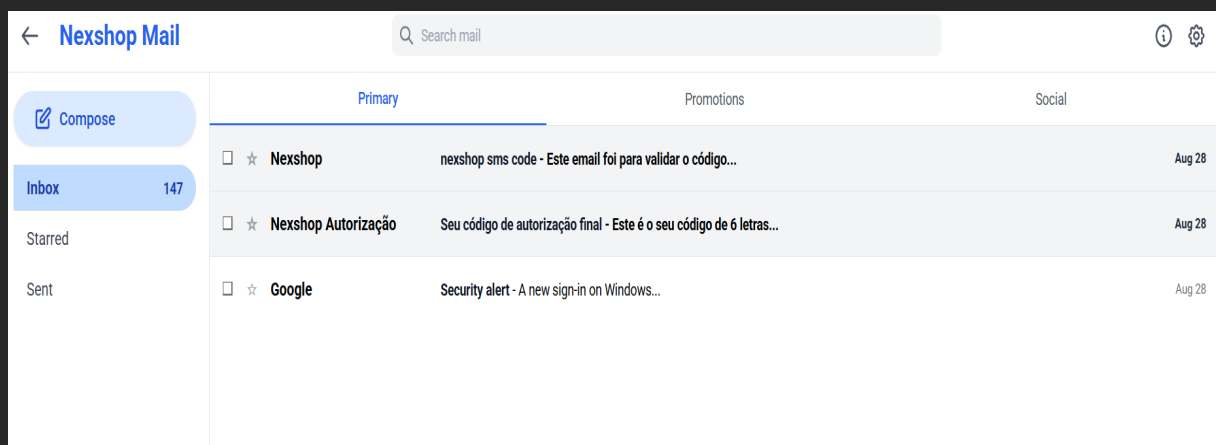


Figura 4 Pagina do Nexshop Mail

## 2.4. Implementação do Fluxo de Registro de Novo Usuário

- **Versão Base (en5.html):** O link #register-link era um placeholder não funcional.
- **Versão Final (challenge 4.html):** O link agora aciona `showSection('register')`, exibindo um formulário completo (#register-section) para criação de conta. Este formulário possui sua própria lógica de validação (para campos não vazios) e, em caso de sucesso, inicia o fluxo de autenticação.



Figura 5 Porta de entrada para a aba do user

**Experimentar Agora**

Figura 6 Entrada para a aba de user

# Nexshop Demo

Sistema Anti-Fraude

## Login de Acesso

Usuário:

E-mail:

Senha:

**Entrar**

---

< [Voltar à Web](#) Não tem uma conta? [Registre-se](#)

Figura 7 Aba de Registre-se

# Nexshop Demo

Sistema Anti-Fraude

## Criar Nova Conta

Usuário:

E-mail:

Senha:

**Criar Conta**

---

[< Voltar ao Login](#)

Figura 8 Validação de um novo user



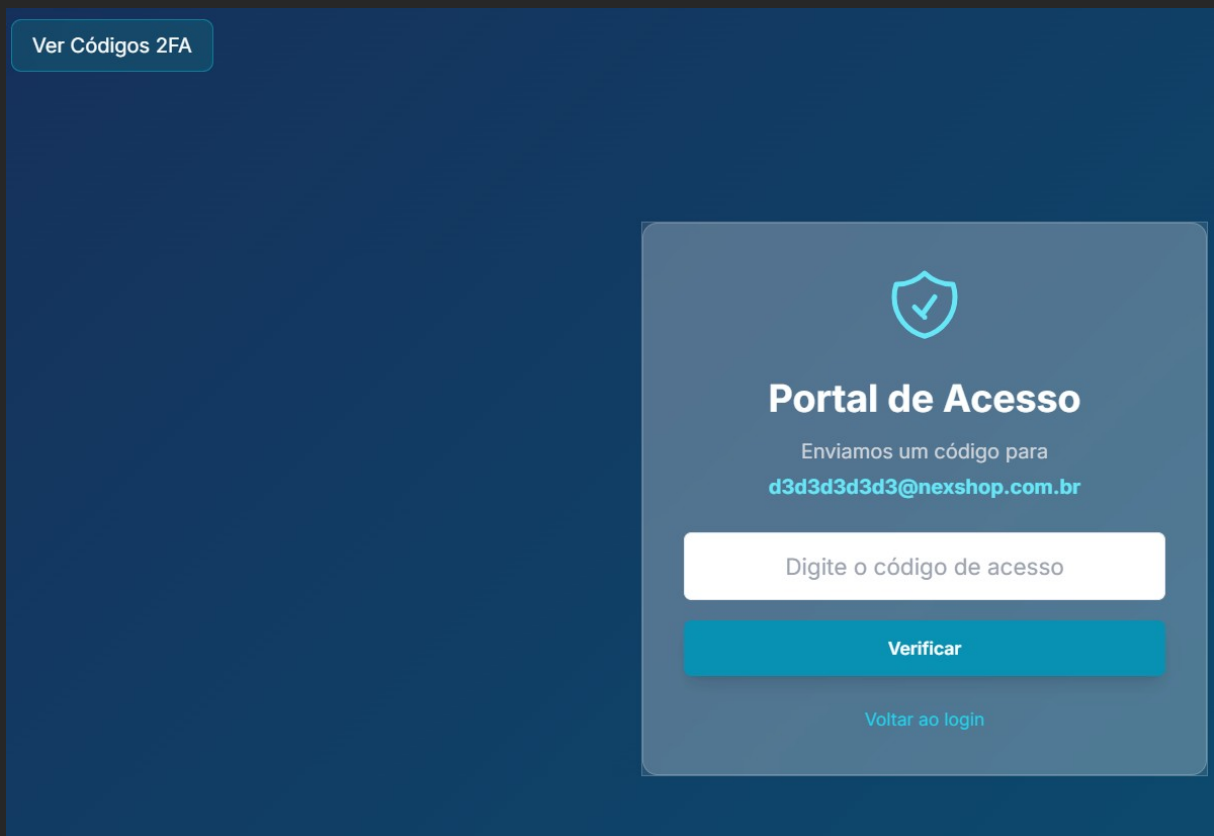


Figura 9 Aba para ver o código de 2fa gerado

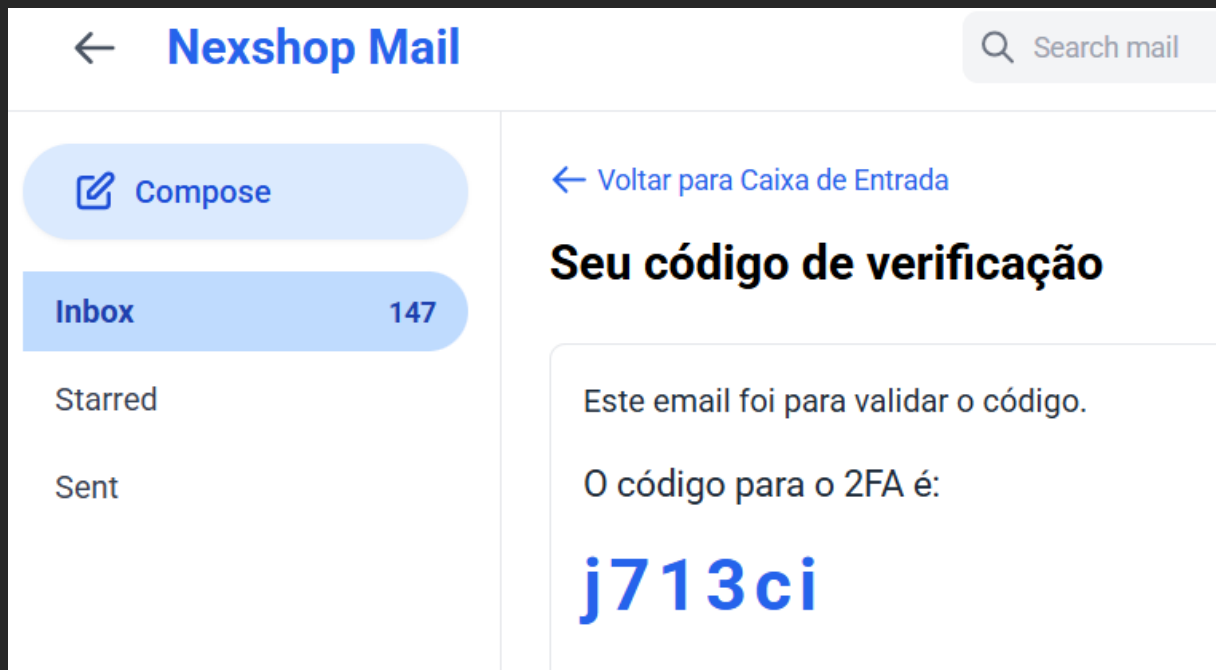


Figura 10 Code 1

## Confirmação de Identidade

Por favor, confirme seus dados e sua identidade.

CPF

111.222.333-44

Telefone

(55)11111-1111



Confirmo que sou humano.

---

**Confirmar**

[Voltar](#)

Figura 11 Validação Padrão

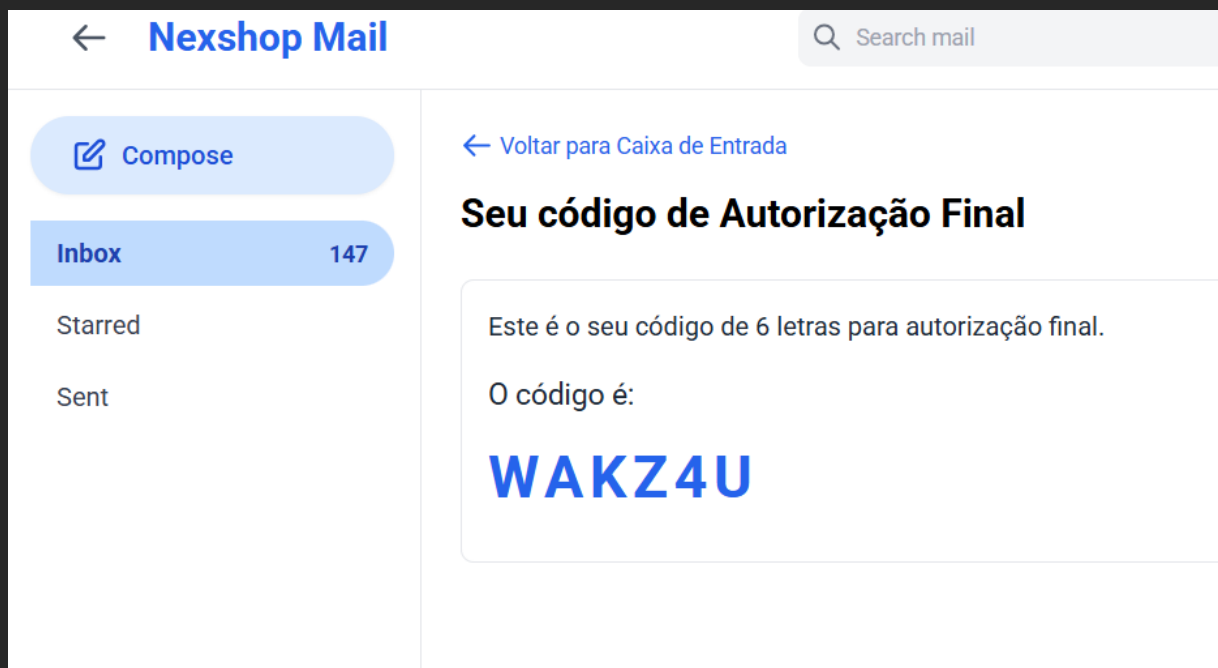


Figura 12 Code 2

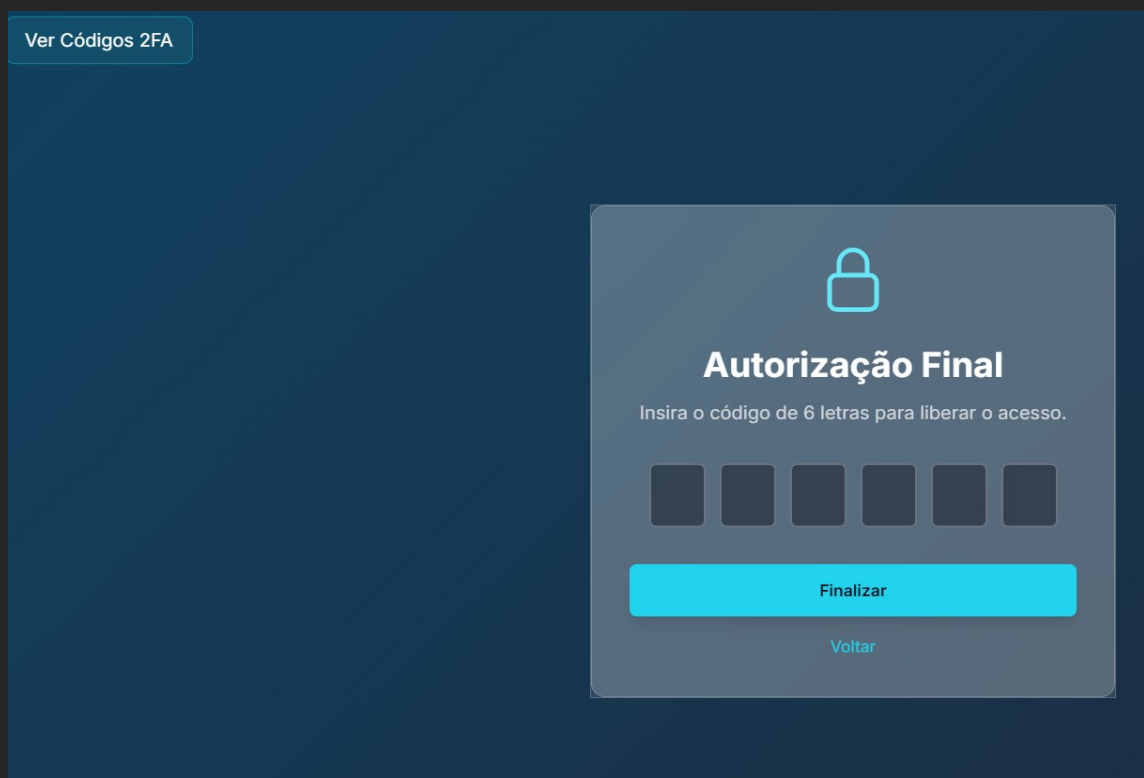


Figura 13 Etapa final

## 2.5. Implementação de Gerenciamento de Estado e Lógica de Validação Dupla

Esta é a modificação de lógica de aplicação mais significativa. Para suportar tanto o fluxo do usuário "Samuel" quanto o novo fluxo de registro, uma variável de gerenciamento de estado foi introduzida: `let validationMode = 'default';`.

- **Definição de Estado:**
  - o O login bem-sucedido como "Samuel" define explicitamente `validationMode = 'default'`.
  - o Um registro bem-sucedido define `validationMode = 'new_user'`.
- **Redefinição de Estado:** O estado é redefinido para 'default' sempre que o usuário retorna à tela inicial, através da função centralizada `goToDownloadScreen()`, que é chamada pelos botões "Reiniciar Demonstração", "Voltar à Web" (do login) e "Voltar" (do Gmail).
- **Lógica Condicional:** Todos os formulários de validação (2fa-form, details-form, phone-code-form) agora contêm blocos condicionais que verificam o valor de `validationMode`:
  - o **Se 'default':** O código aplica as regras estritas (ex: `code === 'samuel'`, `cpfMask.unmaskedValue === '11122233344'`, `code === 'EQUIPE'`).
  - o **Se 'new\_user':** O código aplica regras flexíveis (ex: `code.length > 0`, `cpfMask.unmaskedValue.length > 0`, `code.length === 6`), permitindo que a PoC demonstre o fluxo de ponta a ponta sem exigir dados estáticos.

## 2.6. Análise de Correção Crítica: Validação EQUIPE

- **Versão Base (en5.html):** A lógica de validação para o código final continha um erro: `return (code.toLowerCase() === 'equipe')`.
- **Problema Técnico:** A função `setupFinalCodeInputs` (presente em ambas as versões) força toda a entrada do usuário nesses campos para maiúsculas (`input.value = input.value.toUpperCase();`). Portanto, a variável `code` sempre conteria um valor maiúsculo (ex: "EQUIPE"). Ao forçá-la para minúsculo (`code.toLowerCase()`) antes da comparação, a verificação `=== 'equipe'` sempre funcionaria (assumindo que o usuário digitou "equipe" em qualquer caixa), mas isso estava logicamente incorreto e inconsistente.
- **Versão Final (challenge 4.html):** A lógica foi corrigida para refletir a entrada de dados real e o requisito de maiúsculas. O `.toLowerCase()` foi removido, e a comparação agora verifica `code === 'EQUIPE'`. Isso alinha a validação com a regra de entrada (maiúsculas) e com o código fornecido ao usuário na simulação do Gmail (EQUIPE).

### 3. CONCLUSÃO TÉCNICA

O artefato final (`challenge_4.html`) evolui com sucesso a PoC inicial (`en5.html`) de um script de validação linear para um portal de demonstração de software robusto e interativo. A implementação bem-sucedida de um ponto de entrada (página de produto), a geração dinâmica de um artefato para download (cumprindo o requisito 5.2 do DOCX) e a simulação de serviços externos (Nexshop Mail) criam uma experiência de demonstração de alta fidelidade.

Tecnicamente, a introdução de uma variável de gerenciamento de estado (`validationMode`) para permitir fluxos de validação duplos (estrito e flexível) e a correção da lógica de validação de maiúsculas/minúsculas (EQUIPE) demonstram uma arquitetura de PoC madura, capaz de simular múltiplos cenários de usuário conforme exigido para a validação do projeto NexShop Anti-Fraude.

Link : <https://www.youtube.com/watch?v=IEAB2731bvE>